



Retailer Adding Page — Technical Documentation

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Retailer Adding Page — Technical Documentation	1
Overview	3
Registration & Discovery	3
Authentication & Access Control	4
Database Architecture	6
Column Management	7
Session State	8
Method 1 — Inline Editor	8
Method 2 — Bulk Upload	10
Email Notifications	11
UI Components	12
Error Handling	12
Security Notes	12
Permission Matrix	13
Dependency Graph	13

Application: Market Data Portal (MDP)

Source: `app/pages/retailer_adding.py`

URL path: `/retailer_adding`

UI title: Retailer Manager

Purpose: CRUD and bulk upload for retail company master records in `coreiq_companies`.

Overview

The Retailer Manager page lets authorized users **view, edit, insert, and bulk-upload** rows in the company master table. It is restricted to **local and staging** environments, gated by **page-level Access Control List (ACL)**, and writes an **audit trail** plus **email notifications** on successful mutations.

Registration & Discovery

Property	Value
Registered in <code>main.py</code>	Yes (<code>url_path=retailer_adding</code>)
Navigation link	Footer Quick Links on staging only, for <code>_RETAILER_ALLOWLIST</code> emails (<code>navigation.py</code>)
Header <code>current_page</code>	<code>retailer_manager</code>

Diagram — Part A: Route & auth

Diagram — Part A: Route & auth

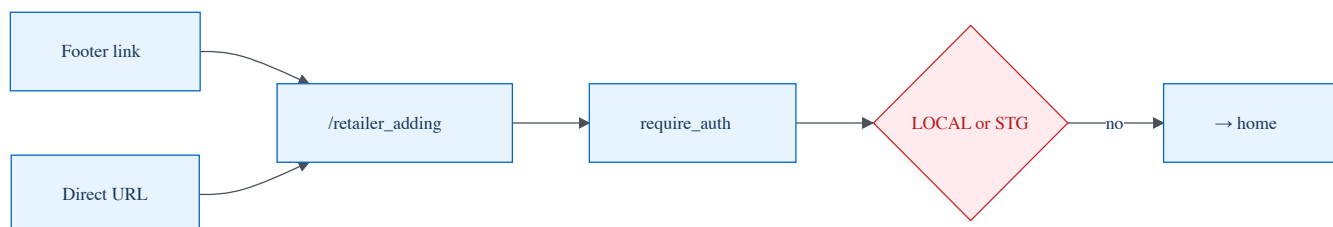
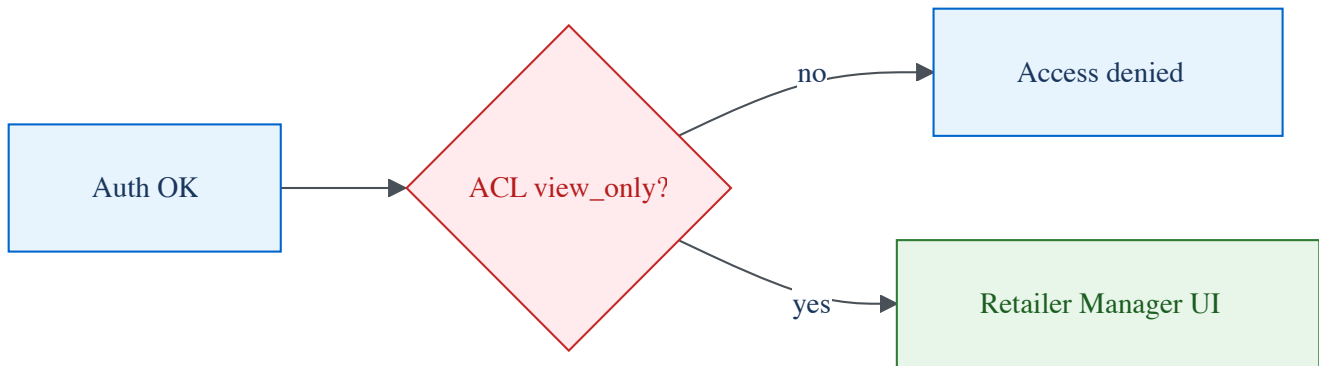


Diagram — Part B: ACL gate

Diagram — Part B: ACL gate



Authentication & Access Control

Auth bootstrap

```

auth_user = require_auth()
user_email = get_current_user()
  
```

Environment guard

```

if config.env not in (Environment.LOCAL, Environment.STAGING):
    st.switch_page("pages/home.py")
  
```

Production users are redirected to Home — the page must not run against prod DB from this entry point.

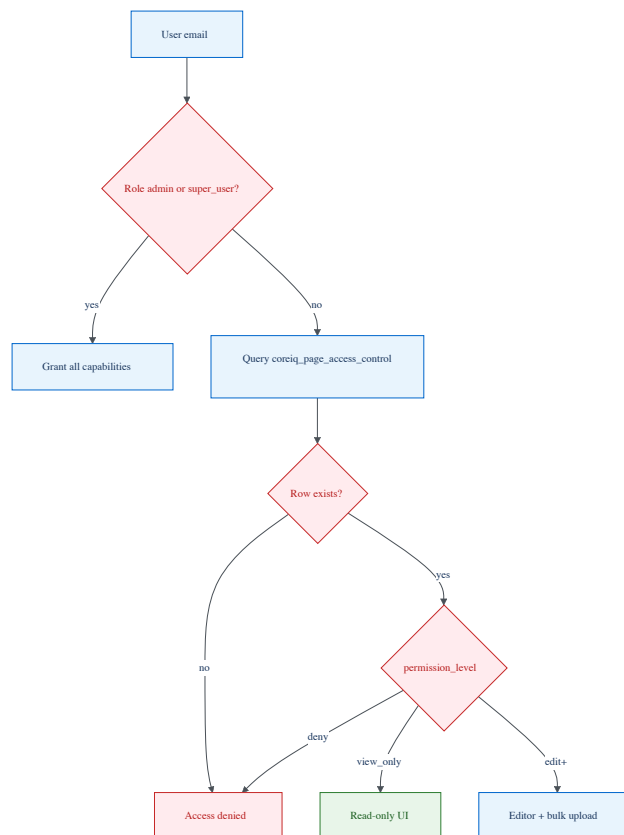
Page ACL (`coreiq_page_access_control`)

Check	Permission required	Effect
Page entry	<code>view_only</code>	Can load page and view data
Edit / save / upload / download	<code>edit</code>	<code>CAN_EDIT_RETAILER = True</code>

```
has_access = AccessControlManager.check_access("retailer_adding", user_email,
"view_only")
CAN_EDIT_RETAILER = AccessControlManager.check_access("retailer_adding", user_email,
"edit")
```

Important: `retailer_adding` has **no default access** — users without an ACL row are denied unless they hold `admin` or `super_user` role (global bypass in `AccessControlManager.check_access`).

Important: retaileradding has no default access — users without an ACL row are denied unless they hold admin or superuser role (global bypass in AccessControlManager.checkaccess).



Footer allowlist (navigation only)

Separate from ACL — controls **link visibility** in staging footer:

`_RETAILER_ALLOWLIST` in `navigation.py` (5 emails including `dataautomation@coresight.com`).

Database Architecture

Connection pattern

Uses **direct** `pymysql` connection — **not** `DatabaseManager` / SQLAlchemy stack used elsewhere.

Env var	Purpose
<code>STG_DB_HOST</code> , <code>STG_DB_PORT</code> , <code>STG_DB_NAME</code>	MySQL target
<code>STG_DB_USER</code> , <code>STG_DB_PASSWORD</code>	Credentials
<code>SSL_CA</code> , <code>ENABLE_SSL</code>	Azure MySQL TLS

Primary table

`coreiq_companies` — company master (tickers, names, industry, exchange, source, etc.)

Business key (unique constraint)

```
(ticker, source, exchange_acronym)
```

Constraint name in comments: `uq_coreiq_companies_name_source`

Audit table

`coreiq_companies_audit_log` — auto-created via `ensure_audit_table()` if missing.

Column	Type	Notes
<code>id</code>	BIGINT AUTO_INCREMENT	PK
<code>event_time</code>	DATETIME	Default CURRENT_TIMESTAMP
<code>user_email</code>	VARCHAR(255)	Actor
<code>action_type</code>	VARCHAR(50)	e.g. UPDATE, INSERT_EDITOR, INSERT_UPLOAD
<code>table_name</code>	VARCHAR(255)	Default <code>coreiq_companies</code>
<code>row_count</code>	INT	Rows affected
<code>status</code>	VARCHAR(50)	SUCCESS / FAILED
<code>details</code>	TEXT	Truncated row summaries
<code>error_message</code>	TEXT	On failure

Reference tables (read-only union export)

Table	Purpose
<code>coreiq_sec_companies_all</code>	SEC company universe
<code>coreiq_non_sec_companies_all</code>	Non-SEC company universe

`build_union_df()` concatenates both with `company_source` = `SEC` / `NON_SEC` for CSV download.

Column Management

System-managed columns (never user-editable)

```
SYSTEM_MANAGED_COLUMNS = {"data_inserted_at"}
```

Also excluded dynamically when:

- `AUTO_INCREMENT` or `GENERATED` in `INFORMATION_SCHEMA.COLUMNS.EXTRA`
- `COLUMN_DEFAULT = CURRENT_TIMESTAMP`

Schema introspection

- `get_table_columns()` — full column metadata from `INFORMATION_SCHEMA`
- `get_primary_key_column()` — PK name (typically `id`)
- `get_pk_and_insertable_columns()` → `(pk, all_cols, upload_cols)`

`upload_cols` = insertable columns minus PK — used for bulk upload template and editor inserts.

Session State

Key	Initial	Purpose
<code>df</code>	On load / refresh	Cached <code>coreiq_companies</code> rows
<code>df_limit</code>	Matches <code>limit</code> input	Invalidates cache when limit changes
<code>editor_save_in_progress</code>	<code>False</code>	Prevents double-submit on Save
<code>bulk_insert_in_progress</code>	<code>False</code>	Prevents double-submit on bulk insert

Widget keys: `retailer_editor` (`st.data_editor`), file uploader, search input.

Method 1 — Inline Editor

Load

```
df = load_data(int(limit)) # SELECT * FROM coreiq_companies LIMIT n
```

Default limit: 500 (UI allows 10–1000).

Search

`filter_editor_df(df, search_text)` — case-insensitive match across all columns (stringified).

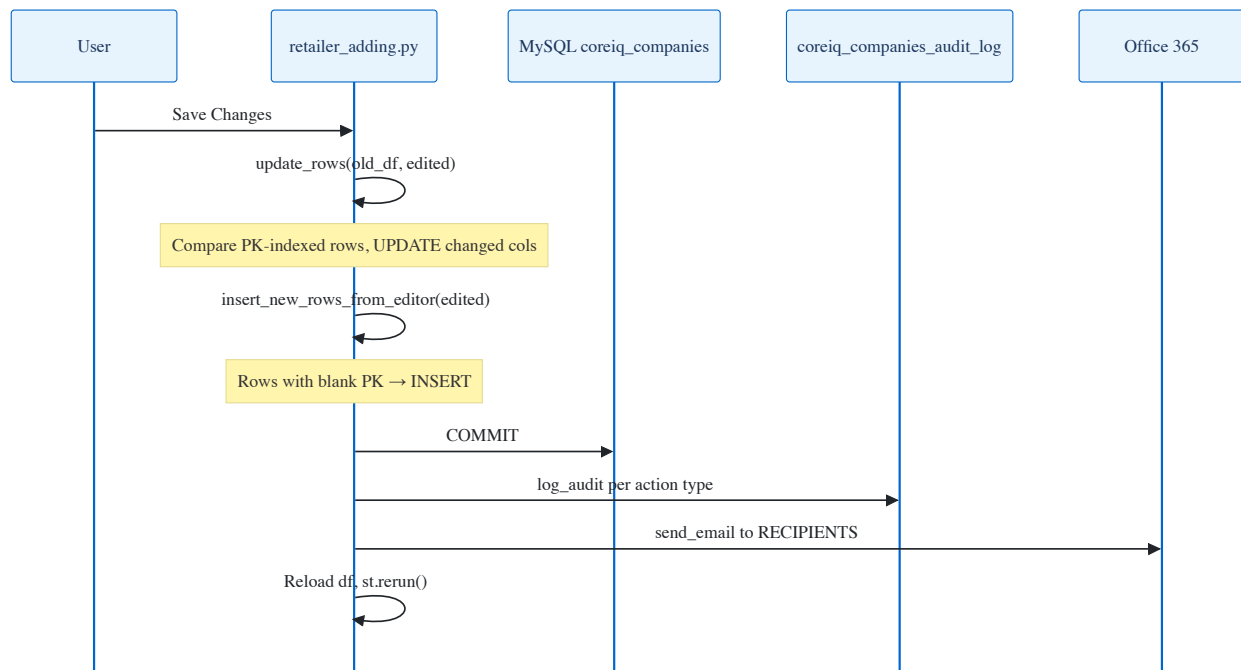
Editor

- `st.data_editor` with `num_rows="dynamic"` when `CAN_EDIT_RETAILER`
- PK column disabled via `build_editor_column_config`
- Read-only `st.dataframe` for view-only users

Save pipeline

Retailer save validates ACL, upserts company rows, and writes an audit log entry.

Retailer save validates ACL, upserts company rows, and writes an audit log entry.



`update_rows(old_df, new_df, pk, upload_cols)`

- Indexes by primary key.
- Builds `SET col=%s` for changed editable columns only.
- Normalizes compare via `normalize_for_compare()` (handles `Timestamp`, `NaN`, strings).

`insert_new_rows_from_editor`

- Selects rows where PK is blank (`is_blank_value`).

- **Deduplicates** within payload by business key (`(ticker, source, exchange_acronym)`).
- Skips rows already in DB (`fetch_existing_business_keys`).
- Catches `IntegrityError` for race / constraint duplicates.

Downloads (edit permission)

Button	Output
Download Screen Data (CSV)	Current edited dataframe
Download All SEC + NON_SEC (CSV)	<code>build_union_df()</code>

Method 2 — Bulk Upload

Supported formats

`.xlsx`, `.csv`

Flow

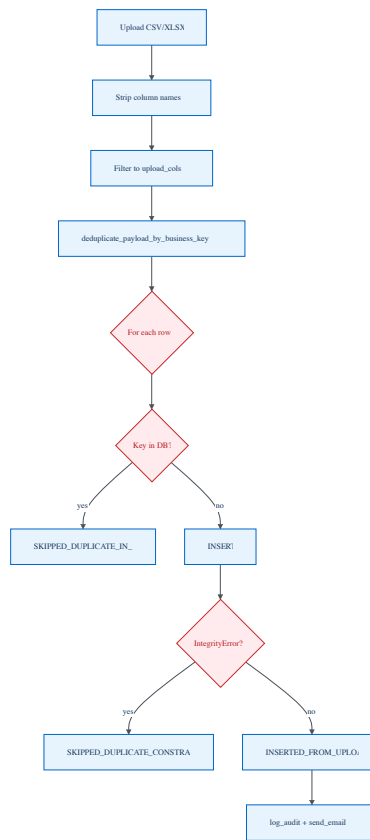
1. User uploads file → preview in `st.dataframe`
2. Invalid columns warned and ignored
3. **Insert Bulk Rows** → `insert_rows(upload_df, upload_cols)`
4. Same dedup / business-key / integrity handling as editor inserts
5. Audit + email on success

Template

Download Bulk Upload Template (CSV) — empty CSV with `upload_cols` headers only.

Bulk CSV upload validates rows, inserts companies, and reports per-row results.

Bulk CSV upload validates rows, inserts companies, and reports per-row results.



Email Notifications

Setting	Value
SMTP	smtp.office365.com:587
From	dataautomation@coresight.com
Password	EMAIL_PASSWORD env
Recipients	shashangupta, RisthaVilinaDsa, PhilipMoore, dataautomation @ coresight.com

Triggered on: UPDATE , INSERT_EDITOR , INSERT_UPLOAD when row count > 0.

Body includes action, row count, performer email, and up to 200 detail lines.

UI Components

Component	Source
<code>hide_sidebar</code> , <code>render_styles</code> , <code>set_page_layout</code>	<code>components.styles</code>
<code>render_header(current_page="retailer_manager")</code>	<code>components.navigation</code>
<code>render_coresight_footer()</code>	<code>components.navigation</code>
Custom <code>.rm-card</code> CSS	Inline in page

`HAS_SHARED_UI` flag — page degrades gracefully if imports fail.

Error Handling

- Save failures → `log_audit(..., status="FAILED", error_message=str(e))`
- Audit logging failures → `st.warning` (non-fatal)
- Email failures → `st.warning` (non-fatal)

Security Notes

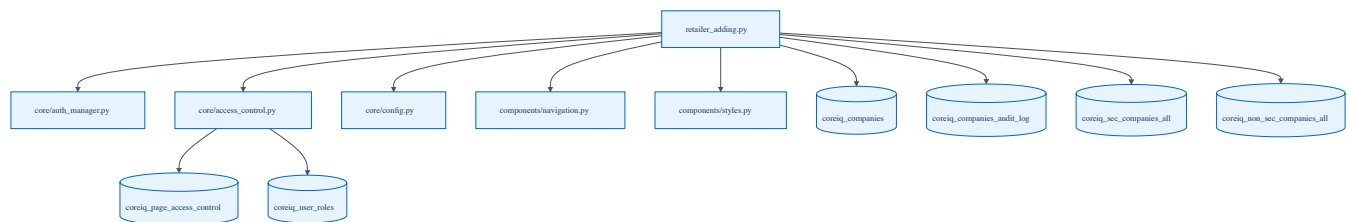
1. **Staging-only env guard** — primary prod safety mechanism.
2. **ACL** — no implicit access; explicit `coreiq_page_access_control` row required (except admin/super_user).
3. **Direct pymysql** — bypasses centralized `db_manager` read/write patterns; uses manual transactions (`autocommit=False`).
4. **No DELETE UI** — users can update and insert only; no row deletion exposed.

Permission Matrix

Role / ACL	View data	Edit rows	Bulk upload	SEC+NON_SEC export
admin / super_user	Yes	Yes	Yes	Yes
view_only	Yes	No	No	No
edit / delete_admin / allow	Yes	Yes	Yes	Yes
No row / deny	No	No	No	No

Dependency Graph

Dependency Graph



Generated from source analysis of the Market Data Portal (MDP). Last reviewed against codebase structure as of project documentation pass.